

MUSTAFACODE

Lecture 20

Modified String Class & Unary Operators (Detailed Notes)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 20: Modified String Class & Unary Operators (Detailed Notes)

Lecture No.20 — Modified String Class & Unary Operators (Detailed Notes)

Introduction

Is lecture mein hum ne ye important concepts seekhe:

- Modified String class
- Character changing techniques
- Subscript `[]` operator overloading
- Function `()` operator overloading
- Substring extraction
- Unary operators overloading

Ye concepts Object Oriented Programming mein bohat important hain kyun ke ye humari custom classes ko built-in data types jaisa powerful bana dete hain.

20.1 Modified String Class

Humari basic String class kuch is tarah hoti hai:

```
class String {  
private:  
    char *bufferPtr;  
    int size;  
  
public:  
    String();  
    String(char *ptr);  
  
    void SetString(char *ptr);  
    const char *GetString();  
};
```

Problem

```
String str2;  
str2.SetString("Ping");
```

Agar hum "Ping" ko "Pong" banana chahein:

```
str2.SetString("Pong");
```

Issue:

- Purani memory delete hoti hai
- Nayi memory allocate hoti hai
- Agar string bari ho to unnecessary overhead hota hai

Better Solution — Sirf Ek Character Change Karna

Puri string replace karne ke bajaye hum sirf ek character change kar sakte hain.

SetChar Function

```
class String {  
private:  
    char *bufferPtr;  
    int size;  
  
public:  
    void SetChar(char c, int pos);  
};
```

Function Definition

```
void String::SetChar(char c, int pos) {  
  
    if (bufferPtr != NULL) {  
  
        if (pos > 0 && pos <= size) {  
  
            bufferPtr[pos - 1] = c;  
        }  
    }  
}
```

Example

```
#include <iostream>  
using namespace std;  
  
int main() {  
  
    String str1("Ping");  
  
    str1.SetChar('o', 2);  
  
    return 0;  
}
```

Output

```
Pong
```

Real Life Example

Socho aap ek notebook mein likhte ho:

```
Ping
```

Agar sirf ek letter change karna ho to puri notebook replace nahi karte, sirf ek letter edit kar dete ho.

20.2 Subscript `[]` Operator

Built-in arrays mein:

```
char array[5] = "Ping";  
array[1] = 'o';
```

Hum chahte hain ke humari String class bhi aise behave kare.

Overloading `[]` Operator

```
class String {  
public:  
    char &operator[](int);  
};
```

Definition

```
char &String::operator[](int pos) {  
  
    return bufferPtr[pos - 1];  
}
```

Why `char &` (Reference)?

Reference return karna zaroori hai kyun ke:

- Assignment possible hota hai
- Original memory access hoti hai

Example

```
#include <iostream>  
using namespace std;  
  
int main() {  
  
    String s1("Ping");  
  
    cout << s1.GetString() << endl;  
  
    s1[2] = 'o';  
  
    cout << s1.GetString();  
  
    return 0;  
}
```

Output

```
Ping  
Pong
```

[] Operator as l-value

```
s1[2] = 'o';
```

Yahan `s1[2]` ek memory location represent kar raha hai.

[] Operator as r-value

```
cout << s1[2];
```

Yahan sirf value read ho rahi hai.

Real Life Example

Jaise address system:

```
House[2]
```

Specific house ko access karta hai.

Waise hi:

```
s1[2]
```

Specific character ko access karta hai.

20.3 Function () Operator

C++ mein hum `()` operator bhi overload kar sakte hain.

Ye bohat flexible operator hota hai.

Features

- Must be member function
 - Multiple parameters le sakta hai
 - Any return type ho sakta hai
 - Generic behavior provide karta hai
-

Example

```
class String {  
public:  
    char &operator()(int);  
};
```

Definition

```
char &String::operator()(int pos) {  
  
    return bufferPtr[pos - 1];  
}
```


Usage

```
#include <iostream>
using namespace std;

int main() {

    String s1("Ping");

    char g = s1(2);

    s1(2) = 'o';

    cout << g << endl;
    cout << s1.GetString();

    return 0;
}
```

Output

```
i
Pong
```

Explanation

Step 1

```
char g = s1(2);
```

Second character `i` variable `g` mein store ho gaya.

Step 2

```
s1(2) = 'o';
```

Second character replace ho gaya.

Difference Between `[]` and `()`

Operator	Purpose
<code>[]</code>	Array-like access
<code>()</code>	Function-like behavior

20.4 Function Operator for Substring

Hum `()` operator ko substring extraction ke liye bhi use kar sakte hain.

Class Declaration

```
class String {  
public:  
    String operator()(int, int);  
};
```

Definition

```
String String::operator()(int index, int subLength) {  
  
    char *ptr = new char[subLength + 1];  
  
    for (int i = 0; i < subLength; i++) {  
  
        ptr[i] = bufferPtr[i + index - 1];  
    }  
  
    ptr[subLength] = '\0';  
  
    String str(ptr);  
  
    delete[] ptr;  
  
    return str;  
}
```

Example

```
#include <iostream>  
using namespace std;  
  
int main() {  
  
    String s("Hello World");  
  
    cout << s(1, 5);  
  
    return 0;  
}
```

Output

```
Hello
```

Explanation

```
s(1,5)
```

Matlab:

- Position 1 se start karo
- 5 characters lo

Real Life Example

Jaise paragraph mein se sirf ek word highlight karna.

20.5 Unary Operators

Unary operators sirf ek operand par work karte hain.

Common Unary Operators

```
& * + - ++ -- ! ~
```

Examples

```
--x  
-(x++)  
!(*ptr++)
```

Prefix vs Postfix

Prefix

```
++x
```

Pehle increment hota hai.

Postfix

```
x++
```

Pehle value use hoti hai phir increment hota hai.

Syntax

Member Function

```
TYPE operator OP();
```

No parameters hotay hain.

Non-Member Function

```
friend TYPE operator OP(TYPE &t);
```

Object parameter ke through pass hota hai.

Overloading Unary Minus (-)

Class

```
class Complex {  
  
    private:  
        float real;  
        float img;  
  
    public:  
        Complex operator-();  
};
```

Definition

```
Complex Complex::operator-() {  
  
    Complex temp;  
  
    temp.real = -real;  
    temp.img = -img;  
  
    return temp;  
}
```

Example

```
#include <iostream>  
using namespace std;  
  
int main() {  
  
    Complex c1(1.0, 2.0);  
    Complex c2;  
  
    c2 = -c1;  
  
    return 0;  
}
```

Output

```
c2.real = -1.0  
c2.img  = -2.0
```

Real Life Example

Bank balance:

```
+500
```

Unary minus ke baad:

```
-500
```

Unary Plus (+)

Unary plus bhi overload hota hai.

Operator	Result
-	Sign reverse
+	Same value

Important Interview Questions

Q1: Why return `char &` in `[]` operator?

- Original memory return hoti hai
 - Assignment possible hota hai
-

Q2: Why is `operator()` called function operator?

Kyuki object function ki tarah behave karta hai:

```
s1(2)
```

Q3: Can `operator()` take multiple arguments?

Yes:

```
s(1,5)
```

Key Points Summary

Concept	Purpose
<code>SetChar()</code>	Single character modify
<code>[]</code> operator	Array-like access
<code>()</code> operator	Function-like behavior
Substring operator	String ka part extract
Unary operators	Single operand operations
Unary <code>-</code>	Sign reverse

Final Conclusion

Is lecture mein hum ne seekha:

- Custom classes ko built-in types jaisa powerful banaya ja sakta hai
- Operator overloading code ko readable aur elegant banati hai
- `[]` aur `()` operators string handling ko easy banate hain
- Unary operators custom objects par mathematical behavior implement karte hain

Ye concepts advanced C++ OOP aur real-world software development mein bohat use hote hain.

MUSTAFA CO